

Check Kafka Connector Status

A command-line utility that queries the Kafka Connect REST API and reports the health of all configured Debezium CDC connectors.

This is the **Theme 1 (Pause-Driven Offset Aging)** edition. The original vendor tool derived health from connector/task *state* only, so a connector that reported **RUNNING** while making no forward progress showed up as **HEALTHY** — the silent-stall masking behavior (issues #16, #17) that turns a pause into multi-day data loss. This version adds **offset-progression detection**, **pause-age awareness**, and **auto-resume**. See [root-cause-analysis.md](#) → Theme 1 → *Tooling Change* for the rationale.

Requirements

- Python 3.7+
- [requests](#) (`pip install requests`)
- Network access to the Kafka Connect REST API
- Kafka Connect **3.6+** for offset-progression detection (the `GET /connectors/{name}/offsets` endpoint). On older Connect the tool degrades gracefully to state-only health.

What's new vs. the original

Capability	Health state	How
Offset-progression detection	STALLED	Compares each source connector's offset (Oracle SCN / MySQL file+pos / SQL Server LSN) across runs. RUNNING + offset frozen past <code>--stall-threshold</code> → STALLED .
Pause-age awareness	AT_RISK	Tracks how long a connector has been PAUSED ; past <code>--retention-warn</code> it escalates to AT_RISK before the offset ages out of source retention.
Auto-resume (prevention)	—	<code>--resume-paused</code> issues <code>POST /connectors/{name}/resume</code> ; dry-run by default.
Monitoring operation	—	<code>--watch/--interval</code> loop and <code>--exit-code</code> for cron / Kubernetes CronJob alerting.

Health states

Health	Meaning
HEALTHY	Connector and all tasks running, offset advancing
STALLED	NEW — reports RUNNING but source offset hasn't advanced past the threshold
AT_RISK	NEW — paused long enough to risk the offset aging out of source retention
FAILED	Connector-level failure
TASK_FAILED	One or more tasks failed
PAUSED	Connector is paused (under the AT_RISK threshold)

Health	Meaning
STOPPED	Connector is stopped
UNASSIGNED	Tasks not yet assigned to a worker

Stateful operation (important)

Stall detection is **stateful across runs** — it compares the current offset to the one captured on the previous run. That snapshot lives in `--state-file` (default `~/.connector_status_state.json`).

- The **same** state file must persist between invocations for **STALLED** / **AT_RISK** to ever trigger.
- A single run can never report **STALLED** (no prior offset to compare). The first run after a fresh state file establishes the baseline.
- For scheduled use, either keep state on a persistent volume (see the CronJob manifest) or run with `--watch` so a single long-lived process holds state.

Usage

```
# Table view (remote host)
python check_kafka_connector_status.py --host https://<connect-host>

# Table view (port-forwarded / local)
python check_kafka_connector_status.py --host http://localhost:8083

# JSON output
python check_kafka_connector_status.py --host http://localhost:8083 --json

# Show only unhealthy connectors (includes STALLED and AT_RISK)
python check_kafka_connector_status.py --host http://localhost:8083 --unhealthy

# Filter to a single state
python check_kafka_connector_status.py --host http://localhost:8083 --filter
STALLED

# Continuous monitoring: poll every 60s, hold offset state in-process
python check_kafka_connector_status.py --host http://localhost:8083 --watch --
interval 60

# Mark RUNNING-but-frozen connectors STALLED after 10 minutes of no offset
movement
python check_kafka_connector_status.py --host http://localhost:8083 --stall-
threshold 600

# Warn (AT_RISK) once a connector has been paused for ~3.5 days
# (set near the source retention window – e.g. MySQL binlog_expire_logs_seconds)
python check_kafka_connector_status.py --host http://localhost:8083 --retention-
warn 302400

# Prevention: report which paused connectors WOULD be resumed (dry-run default)
python check_kafka_connector_status.py --host http://localhost:8083 --resume-
paused

# Prevention: actually resume paused connectors
```

```
python check_kafka_connector_status.py --host http://localhost:8083 --resume-paused --no-dry-run

# Alerting: non-zero exit if anything is
FAILED/TASK_FAILED/STALLED/AT_RISK/UNASSIGNED
python check_kafka_connector_status.py --host http://localhost:8083 --exit-code
--unhealthy
```

CLI arguments

Argument	Default	Description
<code>--host</code>	<i>(required)</i>	Kafka Connect REST API URL
<code>--json</code>	off	Output as JSON instead of a table
<code>--filter STATE</code>	—	Show only connectors in this health state
<code>--unhealthy</code>	off	Show only non- HEALTHY connectors
<code>--timeout</code>	30	Per-request timeout in seconds
<code>--state-file</code>	<code>~/.connector_status_state.json</code>	Where offset snapshots persist between runs
<code>--stall-threshold</code>	300	Seconds of no offset movement before a RUNNING connector is STALLED
<code>--retention-warn</code>	<i>(disabled)</i>	Pause duration (seconds) after which PAUSED → AT_RISK . Set near source retention.
<code>--watch</code>	off	Poll continuously instead of running once
<code>--interval</code>	60	Seconds between polls in <code>--watch</code> mode
<code>--resume-paused</code>	off	Resume PAUSED/AT_RISK connectors
<code>--dry-run / --no-dry-run</code>	dry-run on	With <code>--resume-paused</code> , report vs. actually resume
<code>--exit-code</code>	off	Exit non-zero on FAILED/TASK_FAILED/STALLED/AT_RISK/UNASSIGNED
<code>--no-color</code>	off	Disable ANSI colors in table mode

Exit codes: 0 healthy (or `--exit-code` not set), 1 an unhealthy connector was found (only when `--exit-code` is set), 2 the Connect API was unreachable. The exit status is computed over **all** connectors, not the filtered view, so `--filter` never hides a problem from an alert.

Output fields (table mode)

Column	Description
CONNECTOR	Connector name
TYPE	<code>source</code> or <code>sink</code>
HEALTH	Derived health state (color-coded); <code>STALLED</code> / <code>AT_RISK</code> show elapsed time
TASKS	Running / total task count
FAILED	Number of failed tasks
WORKER	Pod worker ID (shortened)

A summary line shows totals per health state. An `ERRORS:` section lists the first line of any connector/task error trace.

Scheduling

See `k8s/check-kafka-connector-status-cronjob.yaml` for a Kubernetes CronJob that runs the checker on an interval with a persistent volume for the offset state file.